SQL Retail Analytics Project

1. Create Tables

```
CREATE TABLE customers (
customer_id NUMBER,
customer_name VARCHAR2(50)
);

CREATE TABLE stores (
store_id NUMBER,
store_name VARCHAR2(50)
);

CREATE TABLE products (
product_id NUMBER,
category VARCHAR2(50)
);

CREATE TABLE sales (
sale_id NUMBER,
customer_id NUMBER,
store_id NUMBER,
product_id NUMBER,
sale_date DATE,
amount NUMBER,
quantity NUMBER,
discount NUMBER
);
```

2. Insert Data (Auto Generate 1000 Rows)

```
-- Customers
INSERT INTO customers
SELECT LEVEL, 'Cust_'||LEVEL FROM dual CONNECT BY LEVEL<=50;

-- Stores
INSERT INTO stores VALUES (1,'CMR');
INSERT INTO stores VALUES (2,'SR');
```

```
INSERT INTO stores VALUES (3,'Lucky');

-- Products
INSERT INTO products VALUES (1,'Electronics');
INSERT INTO products VALUES (2,'Fashion');
INSERT INTO products VALUES (3,'Grocery');
INSERT INTO products VALUES (4,'Home');

-- Sales (1000 rows)
INSERT INTO sales
SELECT
LEVEL,
MOD(LEVEL,50)+1,
MOD(LEVEL,3)+1,
MOD(LEVEL,4)+1,
DATE '2025-01-01' + MOD(LEVEL,90),
ROUND(DBMS_RANDOM.VALUE(100,10000)),
MOD(LEVEL,5)+1,
CASE WHEN MOD(LEVEL,4)=0 THEN NULL ELSE ROUND(DBMS_RANDOM.VALUE(10,500))
END
FROM dual
CONNECT BY LEVEL <= 1000;
```

1. Data Understanding

Manager Requirement:
Give me overall business performance summary for the last quarter.

Questions:

1. **Find total number of transactions.**

   select count(*) as tot_trans from sales

TOT_TRANS
1000

2. **Find total revenue generated.**

   select sum(amount) as revenue from sales

REVENUE
5038962

3. **Find total unique customers.**

select count(distinct customer_id) as unique_customers from sales

UNIQUE_CUSTOMEI
50

4. **Find average order value.**

select avg(amount) as avg_order_val from sales

AVG_ORDER_VAL
5038.962

5. **Find total quantity sold.**

select sum(quantity) as tot_quant from sales

TOT_QUANT
3000

6. **Find minimum and maximum sale amount.**

select min(amount) as minimum,max(amount) as maximum from sales

MINIMUM	MAXIMUM
102	9975

7. **Find average discount given.**

Select avg(discount) as avg_discount from sales

AVG_DISCOUNT
251.78266666666667

8. **Find total sales per store.**

```
select st.store_name,sum(s.amount) as tot_sales from sales s
join stores st
on s.store_id=st.store_id
group by st.store_name
```

STORE_NAME	TOT_SALES
CMR	1598019
SR	1676247
Lucky	1764696

9. **Find total sales per category.**

```
select p.category,sum(s.amount) as tot_sales from sales s
join products p
on s.product_id=p.product_id
group by p.category
```

CATEGORY	TOT_SALES
Electronics	1329396
Fashion	1174591
Grocery	1205597
Home	1329378

10. **Find number of transactions per day.**

```
select sale_date,count(*) as no_of_trans from sales
group by sale_date
order by sale_date
```

	SALE_DATE	NO_OF_TRANS
1	1/1/2025, 12:00:00	11
2	1/2/2025, 12:00:00	12
3	1/3/2025, 12:00:00	12
4	1/4/2025, 12:00:00	12
5	1/5/2025, 12:00:00	12
6	1/6/2025, 12:00:00	12

2. Data Cleaning

Manager Requirement:
Data looks inconsistent, fix NULL issues before analysis.

Questions:

1. **Replace NULL discount with 0.**

```
Select sale_id,nvl(discount,0) from sales
```

SALE_ID	NVL(DISCOUNT,0)
408	0
409	27
410	351
411	351 53
412	0
413	240
414	345

2. **Find number of NULL discounts.**

Select count(*) from sales where discount is null

COUNT(*)
250

3. **Calculate net revenue (amount - discount).**

Select sale_id,amount-nvl(discount,0) as net_revenue from sales

SALE_ID	NET_REVENUE
408	159
409	7596
410	8053
411	2293
412	5041

4. **Replace NULL quantity with 1.**

Select sale_id,nvl(quantity,1) from sales

SALE_ID	NVL(QUANTITY,1)
408	4
409	5
410	1
411	2
412	3
413	4

5. **Identify rows where amount is NULL.**

Select * from sales where amount is null

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
No items to display.							

6. **Use COALESCE to handle multiple NULL columns.**

Select sale_id,coalesce(discount,0),coalesce(quantity,1) from sales

SALE_ID	COALESCE(DISCOUI	COALESCE(QUANTI
408	0	4
409	27	5
410	351	1
411	53	2
412	0	3
413	240	4

7. Create a cleaned column for revenue.

Select sale_id,amount,discount,amount-nvl(discount,0) as revenue from sales

SALE_ID	AMOUNT	DISCOUNT	REVENUE
408	159	*(null)*	159
409	7623	27	7596
410	8404	351	8053
411	2346	53	2293
412	5041	*(null)*	5041
413	2192	240	1952

8. Check percentage of missing data.

Select
Count(*) as total_rows,
Count(discount) as non_null_dis,
Count(*)-count(discount) as null_dis,
Round(((count(*)-count(discount))/count(*))*100,2) as null_percentage
From sales

TOTAL_ROWS	NON_NULL_DIS	NULL_DIS	NULL_PERCENTAGE
1000	750	250	25

9. Flag rows with missing values.

Select sale_id,
Case when
Discount is null or amount is null or quantity is null then 'Missing data'
Else 'Clean'
End as tag
From sales

SALE_ID	TAG
408	Missing data
409	Clean
410	Clean
411	Clean
412	Missing data

10. Prepare dataset for analysis.

```
Select
sale_id,
customer_id,
store_id,
product_id,
sale_date,
NVL(amount, 0) AS amount,
NVL(quantity, 1) AS quantity,
NVL(discount, 0) AS discount from sales
```

3. Filtering (WHERE)

Manager Requirement:
I want specific insights based on conditions.

Questions:

11. Find sales in last 30 days.

```
Select * from sales where sale_date>=SYSDATE-30
```

12. Find sales above 5000.

```
Select * from sales where amount>5000
```

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
409	10	2	2	2/19/2025,	7623	5	27
410	11	3	3	2/20/2025,	8404	1	351
412	13	2	1	2/22/2025,	5041	3	*(null)*
415	16	2	4	2/25/2025,	7261	1	419
417	18	1	2	2/27/2025,	6596	3	226
418	19	2	3	2/28/2025,	9503	4	374

13. Find sales between 1000 and 5000.

Select * from sales where amount between 1000 and 5000

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
411	12	1	4	2/21/2025,	2346	2	53
413	14	3	2	2/23/2025,	2192	4	240
414	15	1	3	2/24/2025,	3477	5	345
416	17	3	1	2/26/2025,	4886	2	(null)
420	21	1	1	3/2/2025, 1	3931	1	(null)
421	22	2	2	3/3/2025, 1	3032	2	258
423	24	1	4	3/5/2025, 1	2189	4	372

2/21/2025, 12:00:00 AM

14. Find sales from CMR store.

Select s.* from sales s
Join stores st on s.store_id=st.store_id
Where st.store_name='CMR'

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
408	9	1	1	2/18/2025,	159	4	(null)
411	12	1	4	2/21/2025,	2346	2	53
414	15	1	3	2/24/2025,	3477	5	345
417	18	1	2	2/27/2025,	6596	3	226
420	21	1	1	3/2/2025, 1	3931	1	(null)
423	24	1	4	3/5/2025, 1	2189	4	372

15. Find Electronics category sales.

Select s.* from sales s
Join products p on s.product_id=p.product_id
Where p.category='Electronics'

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
408	9	1	1	2/18/2025, 12:00:0(	159	4	(null)
412	13	2	1	2/22/2025, 12:00:0(	5041	3	(null)
416	17	3	1	2/26/2025, 12:00:0(	4886	2	(null)
420	21	1	1	3/2/2025, 12:00:00	3931	1	(null)
424	25	2	1	3/6/2025, 12:00:00	1327	5	(null)
428	29	3	1	3/10/2025, 12:00:0(	8231	4	(null)
432	33	1	1	3/14/2025, 12:00:0(	7917	3	(null)

16. Find sales excluding Grocery category.

Select s.* from sales s
Join products p on s.product_id=p.product_id
Where p.category <> 'Grocery'

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
408	9	1	1	2/18/2025,	159	4	(null)
409	10	2	2	2/19/2025,	7623	5	27
411	12	1	4	2/21/2025,	2346	2	53
412	13	2	1	2/22/2025,	5041	3	(null)
413	14	3	2	2/23/2025,	2192	4	240
415	16	2	4	2/25/2025,	7261	1	419

159

17. Find customers from specific store.

```
select distinct c.customer_name from sales s
join customers c on s.customer_id=c.customer_id
join stores st on s.store_id=st.store_id
where st.store_name='CMR'
```

CUSTOMER_NAME
Cust_9
Cust_12
Cust_15
Cust_18
Cust_21
Cust_24

18. Find high discount transactions.

```
Select * from sales where discount>20
```

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
409	10	2	2	2/19/2025,	7623	5	27
410	11	3	3	2/20/2025,	8404	1	351
411	12	1	4	2/21/2025,	2346	2	53
413	14	3	2	2/23/2025,	2192	4	240
414	15	1	3	2/24/2025,	2192 7	5	345
415	16	2	4	2/25/2025,	7261	1	419

19. Find low quantity sales.

```
Select * from sales where quantity<2
```

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
410	11	3	3	2/20/2025,	8404	1	351
415	16	2	4	2/25/2025,	7261	1	419
420	21	1	1	3/2/2025, 1	3931	1	*(null)*
425	26	3	2	3/7/2025, 1	6517	1	272
430	31	2	3	3/12/2025,	5802	1	370

20. Find recent transactions.

```
select * from sales order by sale_date desc
```

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
359	10	3	4	3/31/2025,	5645	5	390
179	30	3	4	3/31/2025,	9298	5	186
269	20	3	2	3/31/2025,	3645	5	367
989	40	3	2	3/31/2025,	7108	5	117
89	40	3	2	3/31/2025,	1477	5	168
539	40	3	4	3/31/2025,	5147	5	196

4. Aggregation (GROUP BY)

Manager Requirement:
Break down performance across business segments.

Questions:

21. Revenue per store.

```
Select st.store_name,sum(s.amount) from sales s
Join stores st on s.store_id=st.store_id
 Group by st.store_name
```

STORE_NAME	SUM(S.AMOUNT)
CMR	1598019
SR	1676247
Lucky	1764696

22. Revenue per category.

```
SELECT p.category, SUM(s.amount) AS total_revenue
FROM sales s
 JOIN products p ON s.product_id = p.product_id
GROUP BY p.category
```

CATEGORY	TOTAL_REVENUE
Electronics	1329396
Fashion	1174591
Grocery	1205597
Home	1329378

23. Revenue per customer.

```
select c.customer_name,sum(s.amount) from sales s
join customers c on s.customer_id=c.customer_id
```

group by c.customer_name

CUSTOMER_NAME	SUM(S.AMOUNT)
Cust_9	84244
Cust_10	113636
Cust_11	113636 99282
Cust_12	116311
Cust_13	104171
Cust_14	73144

24. Monthly revenue.

select to_char(sale_date,'YYYY-MM') as month, sum(amount) from sales
group by to_char(sale_date,'YYYY-MM')
order by month

MONTH	SUM(AMOUNT)
2025-01	1758650
2025-02	1590879
2025-03	1689433

25. Daily revenue.

select sale_date,sum(amount) from sales group by sale_date

SALE_DATE	SUM(AMOUNT)
2/18/2025, 12:00:00 AM	50129
2/19/2025, 12:00:00 AM	78520
2/20/2025, 12:00:00 AM	64755
2/21/2025, 12:00:00 AM	64797
2/22/2025, 12:00:00 AM	63524
2/23/2025, 12:00:00 AM	49387

26. Average sales per store.

Select st.store_name,avg(s.amount) from sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name

STORE_NAME	AVG(S.AMOUNT)
CMR	4798.855855855856
SR	5018.703592814371
Lucky	5299.387387387387

27. Total quantity per category.

```
SELECT p.category, SUM(s.quantity) AS total_revenue
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category
```

CATEGORY	TOTAL_REVENUE
Electronics	750
Fashion	750
Grocery	750
Home	750

28. Total discount per store.

```
Select st.store_name,sum(s.discount) from sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
```

STORE_NAME	SUM(S.DISCOUNT)
CMR	61789
SR	64514
Lucky	62534

29. Customer-wise total spending.

```
select c.customer_name,sum(s.amount) from sales s
join customers c on s.customer_id=c.customer_id
group by c.customer_name
```

CUSTOMER_NAME	SUM(S.AMOUNT)
Cust_9	84244
Cust_10	113636
Cust_11	113636
Cust_12	116311
Cust_13	104171
Cust_14	73144

30. Store-wise transaction count.

```
Select st.store_name,count(*) from sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
```

STORE_NAME	COUNT(*)
CMR	333
SR	334
Lucky	333

5. HAVING

Manager Requirement:
Find top performing segments.

Questions:

31. Customers with revenue > 20000.

```
select c.customer_name,sum(s.amount) from sales s
join customers c on s.customer_id=c.customer_id
group by c.customer_name
having sum(s.amount)>20000
```

CUSTOMER_NAME	SUM(S.AMOUNT)
Cust_9	84244
Cust_10	113636
Cust_11	99282
Cust_12	116311
Cust_13	104171

32. Stores with revenue > 100000.

```
Select st.store_name,sum(s.amount) from sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
Having sum(s.amount)>100000
```

STORE_NAME	SUM(S.AMOUNT)
CMR	1598019
SR	1676247
Lucky	1764696

33. Categories with avg sales > 3000.

```
SELECT p.category, AVG(s.amount) AS avg_sales
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category
```

HAVING AVG(s.amount) > 3000

CATEGORY	AVG_SALES
Electronics	5317.584
Fashion	4698.364
Grocery	4822.388
Home	5317.512

34. Customers with more than 5 transactions.

```
select c.customer_name,count(*) from sales s
join customers c on s.customer_id=c.customer_id
group by c.customer_name
having count(*)>5
```

CUSTOMER_NAME	COUNT(*)
Cust_9	20
Cust_10	20
Cust_11	20
Cust_12	20
Cust_13	20
Cust_14	20

35. Stores with high discount usage.

```
Select st.store_name,sum(s.discount) from sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
Having sum(s.discount)>5000
```

STORE_NAME	SUM(S.DISCOUNT)
CMR	61789
SR	64514
Lucky	62534

36. Categories with low performance.

```
SELECT p.category, SUM(s.amount) AS avg_sales
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category
HAVING SUM(s.amount) < 20000
```

CATEGORY	AVG_SALES
No items to display.	

37. Customers with high avg order value.

```
select c.customer_name,avg(s.amount) from sales s
```

```
join customers c on s.customer_id=c.customer_id
group by c.customer_name
 having avg(s.amount)>4000
```

CUSTOMER_NAME	AVG(S.AMOUNT)
Cust_9	4212.2
Cust_10	5681.8
Cust_11	4964.1
Cust_12	5815.55
Cust_13	5208.55

38. Find top 5 customers using HAVING.

```
 select c.customer_name,sum(s.amount) as revenue from sales s
join customers c on s.customer_id=c.customer_id
group by c.customer_name
order by revenue desc
 fetch first 5 rows only
```

CUSTOMER_NAME	REVENUE
Cust_19	135669
Cust_36	126737
Cust_18	119929
Cust_12	116311
Cust_23	115118

39. Stores with max transactions.

```
 Select st.store_name,count(*) as max_trans from sales s
 Join stores st on s.store_id=st.store_id
Group by st.store_name
 Having count(*)=(
                Select max(cnt)
                From (
                        Select count(*) as cnt from sales
                        Group by store_id
                )
 );
```

STORE_NAME	MAX_TRANS
SR	334

40. Categories with high growth.

```
SELECT p.category,
SUM(CASE WHEN s.sale_date >= SYSDATE - 30 THEN s.amount ELSE 0 END) AS recent_sales,
SUM(CASE WHEN s.sale_date < SYSDATE - 30 THEN s.amount ELSE 0 END) AS old_sales
FROM sales s
JOIN products p ON s.product_id = p.product_id
GROUP BY p.category
HAVING SUM(CASE WHEN s.sale_date >= SYSDATE - 30 THEN s.amount ELSE 0 END) >  SUM(CASE WHEN s.sale_date < SYSDATE - 30 THEN s.amount ELSE 0 END);
```

CATEGORY	RECENT_SALES	OLD_SALES

No items to display.

6. CASE WHEN

Manager Requirement:
Segment customers for business decisions.

Questions:

41. Classify customers as High/Medium/Low spenders.

```
SELECT c.customer_name,
SUM(s.amount) AS total_spent,
CASE
    WHEN SUM(s.amount) > 20000 THEN 'High'
    WHEN SUM(s.amount) BETWEEN 10000 AND 20000 THEN 'Medium'
    ELSE 'Low'
  END AS spender_type
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
```

GROUP BY c.customer_name;

CUSTOMER_NAME	TOTAL_SPENT	SPENDER_TYPE
Cust_9	84244	High
Cust_10	113636	High
Cust_11	99282	High
Cust_12	116311	High
Cust_13	104171	High
Cust_14	73144	High
Cust_15	107671	High

42. Classify transactions as Big/Small.

```
Select sale_id,amount,
Case
        When amount>5000 then 'Big'
        Else 'Small'
End as trans_type
From sales
```

SALE_ID	AMOUNT	TRANS_TYPE
408	159	Small
409	7623	Big
410	8404	Big
411	2346	Small
412	5041	Big
413	2192	Small

43. Create discount flag.

```
Select sale_id,discount
Case
        When discount is null then 'No Discount'
        Else 'Discount Applied'
End as dis_flag
From sales
```

SALE_ID	DISCOUNT	DIS_FLAG
408	*(null)*	No Discount
409	27	Discount Applied
410	351	Discount Applied
411	53	Discount Applied
412	*(null)*	No Discount

44. Segment stores based on revenue.

```
Select st.store_name, sum(s.amount) as tot_revenue,
```

```
Case
        When sum(s.amount)>100000 then 'Top Store'
        When sum(s.amount) between 50000 and 100000 then 'Mid Store'
        Else 'Low Store'
End as store_segment
From sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
```

STORE_NAME	TOT_REVENUE	STORE_SEGMENT
CMR	1598019	Top Store
SR	1676247	Top Store
Lucky	1764696	Top Store

45. Categorize customers based on frequency.

```
Select c.customer_name,
    count(*) AS total_orders,
    Case
      When count(*) > 10 then 'Frequent'
      When count(*) between 5 and 10 then 'Regular'
      Else 'Occasional'
    End as customer_type
From sales s
Join customers c on s.customer_id = c.customer_id
Group by c.customer_name;
```

CUSTOMER_NAME	TOTAL_ORDERS	CUSTOMER_TYPE
Cust_9	20	Frequent
Cust_10	20	Frequent
Cust_11	20	Frequent
Cust_12	20	Frequent
Cust_13	20	Frequent
Cust_14	20	Frequent

46. Create sales buckets.

```
Select sale_id,amount,
Case
        When amount<1000 then 'Low'
        When amount between 1000 and 5000 then 'Medium'
        Else 'High'
End as sales_buckets
```

From sales

SALE_ID	AMOUNT	SALES_BUCKETS
408	159	Low
409	7623	High
410	8404	High
411	2346	Medium
412	5041	High
413	2192	Medium
414	3477	Medium

47. Identify premium customers.

```
SELECT c.customer_name,
    SUM(s.amount) AS total_spent,
    CASE
      WHEN SUM(s.amount) > 30000 THEN 'Premium'
      ELSE 'Normal'
    END AS customer_tag
FROM sales s
JOIN customers c ON s.customer_id = c.customer_id
GROUP BY c.customer_name;
```

CUSTOMER_NAME	TOTAL_SPENT	CUSTOMER_TAG
Cust_9	84244	Premium
Cust_10	113636	Premium
Cust_11	99282	Premium
Cust_12	116311	Premium
Cust_13	104171	Premium
Cust_14	73144	Premium

48. Mark high-value transactions.

```
Select sale_id,amount,
Case
        When amount>10000 then 'High Value'
        Else 'Normal'
End as value_flag
From sales
```

SALE_ID	AMOUNT	VALUE_FLAG
408	159	Normal
409	7623	Normal
410	8404	Normal
411	2346	Normal
412	5041	Normal
413	2192	Normal

49. Classify categories based on performance.

```
Select p.category, sum(s.amount) as tot_revenue,
Case
        When sum(s.amount)>50000 then 'High Performing'
        When sum(s.amount) between 20000 and 50000 then 'Average'
        Else 'Low Performing'
End as performance_tag
From sales s
Join products p on s.product_id=p.product_id
Group by p.category
```

CATEGORY	TOT_REVENUE	PERFORMANCE_TA
Electronics	1329396	High Performing
Fashion	1174591	High Performing
Grocery	1205597	High Performing
Home	1329378	High Performing

50. Create custom labels for reporting.

```
Select sale_id,amount,
CASE
    WHEN amount > 5000 AND discount > 0 THEN 'High Value + Discount'
    WHEN amount > 5000 THEN 'High Value'
    WHEN discount > 0 THEN 'Discount Sale'
    ELSE 'Regular Sale'
  END AS report_label
FROM sales
```

SALE_ID	AMOUNT	REPORT_LABEL
408	159	Regular Sale
409	7623	High Value + Discour
410	8404	High Value + Discour
411	2346	Discount Sale
412	5041	High Value
413	2192	Discount Sale

7. JOINS (Multiple Tables)

Manager Requirement:
Combine customer, product, and store data for insights.

Questions:

51. Join sales with customers table.

Select s.*,c.customer_name from sales s
Join customers c on s.customer_id=c.customer_id

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT	CUSTOMER_NAME
408	9	1	1	2/18/2025, 1	159	4	(null)	Cust_9
409	10	2	2	2/19/2025, 1	7623	5	27	Cust_10
410	11	3	3	2/20/2025, 1	8404	1	351	Cust_11
411	12	1	4	2/21/2025, 1	2346	2	53	Cust_12
412	13	2	1	2/22/2025, 1	5041	3	(null)	Cust_13
413	14	3	2	2/23/2025, 1	2192	4	240	Cust_14

52. Join sales with stores table.

Select s.*,st.store_name from sales s
Join stores st on s.store_id=st.store_id

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT	STORE_NAME
408	9	1	1	2/18/2025,	159	4	(null)	CMR
409	10	2	2	2/19/2025,	159 7623	5	27	SR
410	11	3	3	2/20/2025,	8404	1	351	Lucky
411	12	1	4	2/21/2025,	2346	2	53	CMR
412	13	2	1	2/22/2025,	5041	3	(null)	SR
413	14	3	2	2/23/2025,	2192	4	240	Lucky

53. Join sales with products table.

Select s.*,p.category from sales s
Join products p on s.product_id=p.product_id

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT	CATEGORY
408	9	1	1	2/18/2025,	159	4	(null)	Electronics
409	10	2	2	2/19/2025,	7623	5	27	Fashion
410	11	3	3	2/20/2025,	8404	1	351	Grocery
411	12	1	4	2/21/2025,	2346	2	53	Home
412	13	2	1	2/22/2025,	5041	3	(null)	Electronics
413	14	3	2	2/23/2025,	2192	4	240	Fashion

54. Find customer name with store name and sales.

Select c.customer_name,st.store_name,s.amount
From sales s
Join customers c on s.customer_id=c.customer_id

Join stores st on s.store_id=st.store_id

CUSTOMER_NAME	STORE_NAME	AMOUNT
Cust_9	CMR	159
Cust_10	SR	7623
Cust_11	Lucky	8404
Cust_12	CMR	2346
Cust_13	SR	5041
Cust_14	Lucky	2192

55. Find category-wise revenue using joins.

Select p.category,sum(s.amount) from sales s
Join products p on s.product_id=p.product_id
Group by p.category

CATEGORY	SUM(S.AMOUNT)
Electronics	1329396
Fashion	1174591
Grocery	1205597
Home	1329378

56. Find store-wise customer count.

Select st.store_name,count(distinct s.customer_id) as cust_cnt
From sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name

STORE_NAME	CUST_CNT
CMR	50
SR	50
Lucky	50

57. Find top product per store.

Select store_name,category,total_sales
From (
    Select st.store_name,p.category,sum(s.amount) as total_sales,

```
        RANK() OVER(partition by st.store_name order by sum(s.amount) desc) as
rnk
        From sales s
        Join stores st on s.store_id=st.store_id
        Join products p on s.product_id=p.product_id
        Group by st.store_name,p.category
)
Where rnk=1
```

STORE_NAME	CATEGORY	TOTAL_SALES
CMR	Electronics	421508
Lucky	Home	481102
SR	Lucky me	452273

58. Find customers who purchased from multiple stores.

```
Select c.customer_name from sales s
Join customers c on s.customer_id=c.customer_id
Group by c.customer_name
Having count(distinct s.store_id)>1
```

CUSTOMER_NAME
Cust_9
Cust_10
Cust_11
Cust_12
Cust_13
Cust_14

59. Find sales with product category.

```
Select s.sale_id,s.amount,p.category from sales s
```

Join products p on s.product_id=p.product_id

SALE_ID	AMOUNT	CATEGORY
408	159	Electronics
409	7623	Fashion
410	8404	Grocery
411	2346	Home
412	5041	Electronics
413	2192	Fashion

60. Find revenue per customer per store.

Select c.customer_name,st.store_name,sum(s.amount) from sales s
Join customers c on s.customer_id=c.customer_id
Join stores st on s.store_id=st.store_id
Group by c.customer_name,st.store_name

CUSTOMER_NAME	STORE_NAME	SUM(S.AMOUNT)
Cust_9	CMR	11199
Cust_10	SR	39525
Cust_11	Lucky	30910
Cust_12	CMR	19059
Cust_13	SR	37252
Cust_14	Lucky	16657

61. Find customers with no transactions (LEFT JOIN).

Select c.customer_name from customers c
Left join sales s on c.customer_id=s.customer_id
Where s.sale_id is null

CUSTOMER_NAME

No items to display.

62. Find missing product mappings.

Select s.sale_id from sales s
Left join products p on s.product_id = p.product_id
Where p.product_id is null

SALE_ID

No items to display.

63. Find store performance using joins.

```
Select st.store_name, sum(s.amount) as revenue, count(*) as trans
From sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
```

STORE_NAME	REVENUE	TRANS
CMR	1598019	333
SR	1676247	334
Lucky	1764696	333

64. Find cross-category purchases. (Customers buying multiple categories)

```
Select c.customer_name from sales s
Join customers c on s.customer_id=c.customer_id
Group by c.customer_name
Having count(distinct s.product_id)>1
```

CUSTOMER_NAME
Cust_9
Cust_10
Cust_11
Cust_12
Cust_13
Cust_14

65. Find total revenue using joins.

```
Select sum(s.amount) as tot_revenue from sales s
Join customers c on s.customer_id=c.customer_id
Join stores st on s.store_id=st.store_id
Join products p on s.product_id=p.product_id
```

TOT_REVENUE
5038962

8. Window Functions

Manager Requirement:
Provide advanced analytics without losing row-level data.

Questions:

66. Calculate total spend per customer using window function.

```
Select s.sale_id,c.customer_name,s.amount,
```

```
Sum(s.amount) over(partition by s.customer_id) as total_spend
From sales s
Join customers c on s.customer_id=c.customer_id
```

SALE_ID	CUSTOMER_NAME	AMOUNT	TOTAL_SPEND
450	Cust_1	5379	92991
500	Cust_1	5075	92991
550	Cust_1	2326	92991
600	Cust_1	2740	92991
650	Cust_1	5868	92991
700	Cust_1	9178	92991

67. Rank customers by revenue.

```
Select c.customer_name,sum(s.amount) as revenue,
RANK() OVER(order by sum(s.amount) desc) as rnk
From sales s
Join customers c on s.customer_id=c.customer_id
Group by c.customer_name
```

CUSTOMER_NAME	REVENUE	RNK
Cust_19	135669	1
Cust_36	126737	2
Cust_18	119929	3
Cust_12	116311	4
Cust_23	115118	5
Cust_29	113904	6
Cust_10	113636	7

68. Find top customer per store.

```
Select *
From (
  Select st.store_name, c.customer_name,
      Sum(s.amount) as total_spend,
      RANK() OVER (partition by st.store_name order by sum(s.amount) desc) as
rnk
  From sales s
  Join customers c on s.customer_id = c.customer_id
  Join stores st on s.store_id = st.store_id
  Group by st.store_name, c.customer_name
)
```

STORE	SALE_ID	SALE_DATE	AMOUNT	RUNNING_TOTAL
CMR	360	1/1/2025, 12:00:00	6954	6954
Lucky	180	1/1/2025, 12:00:00	3099	10053
SR	270	1/1/2025, 12:00:00	3535	13588
	90	1/1/2025, 12:00:00	4892	18480
	900	1/1/2025, 12:00:00	3757	22237
	630	1/1/2025, 12:00:00	8871	31108
	720	1/1/2025, 12:00:00	9834	40942

Where rnk = 1;

69. Calculate running total of sales.

```
Select sale_id, sale_date, amount,
    Sum(amount) OVER (order by sale_date ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS running_total
FROM sales;
```

70. Find previous sale using LAG.

```
Select sale_id,sale_date,amount,
LAG(amount) OVER(order by sale_date ) as prev_sale
From sales
```

SALE_ID	SALE_DATE	AMOUNT	PREV_SALE
360	1/1/2025, 12:00:00	6954	*(null)*
180	1/1/2025, 12:00:00	3099	6954
270	1/1/2025, 12:00:00	3535	3099
90	1/1/2025, 12:00:00	4892	3535
900	1/1/2025, 12:00:00	3757	4892
630	1/1/2025, 12:00:00	8871	3757
720	1/1/2025, 12:00:00	9834	8871

71. Find next sale using LEAD.

```
Select sale_id,sale_date,amount,
LEAD(amount) OVER(order by sale_date ) as next_sale
```

From sales

SALE_ID	SALE_DATE	AMOUNT	NEXT_SALE
360	1/1/2025, 12:00:00	6954	3099
180	1/1/2025, 12:00:00	3099	3535
270	1/1/2025, 12:00:00	3535	4892
90	1/1/2025, 12:00:00	4892	3757
900	1/1/2025, 12:00:00	3757	8871
630	1/1/2025, 12:00:00	8871	9834

72. Calculate sales growth.

Select sale_id,sale_date,amount,
Amount-LAG(amount) OVER(order by sale_date) as growth
From sales

SALE_ID	SALE_DATE	AMOUNT	GROWTH
360	1/1/2025, 12:00:00	6954	*(null)*
180	1/1/2025, 12:00:00	3099	-3855
270	1/1/2025, 12:00:00	3535	436
90	1/1/2025, 12:00:00	4892	1357
900	1/1/2025, 12:00:00	3757	-1135
630	1/1/2025, 12:00:00	8871	5114
720	1/1/2025, 12:00:00	9834	963

73. Find difference between transactions.

Select sale_id,amount,
Amount-LAG(amount) OVER(order by sale_date) as diff
From sales

SALE_ID	AMOUNT	DIFF
360	6954	*(null)*
180	3099	-3855
270	3535	436
90	4892	1357
900	3757	-1135
630	8871	5114

74. Calculate cumulative revenue.

```
Select sale_id, sale_date, amount,
    Sum(amount) OVER (order by sale_date ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS cumulative_revenue
From sales
```

SALE_ID	SALE_DATE	AMOUNT	CUMULATIVE_REVE
360	1/1/2025, 12:00:00	6954	6954
180	1/1/2025, 12:00:00	3099	10053
270	1/1/2025, 12:00:00	3535	13588
90	1/1/2025, 12:00:00	4892	18480
900	1/1/2025, 12:00:00	3757	22237
630	1/1/2025, 12:00:00	8871	31108

75. Find moving average of sales.

```
Select sale_id, sale_date, amount,
    avg(amount) OVER (order by sale_date ROWS BETWEEN 2 PRECEDING AND
CURRENT ROW) AS moving_avg
From sales
```

SALE_ID	SALE_DATE	AMOUNT	MOVING_AVG
360	1/1/2025, 12:00:00	6954	6954
180	1/1/2025, 12:00:00	3099	5026.5
270	1/1/2025, 12:00:00	3535	4529.333333333333
90	1/1/2025, 12:00:00	4892	3842
900	1/1/2025, 12:00:00	3757	4061.3333333333335
630	1/1/2025, 12:00:00	8871	5840
720	1/1/2025, 12:00:00	9834	7487.333333333333

76. Rank stores by revenue.

```
 Select st.store_name,sum(s.amount) as revenue,
RANK() OVER(order by sum(s.amount) desc) as rnk
From sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
```

STORE_NAME	REVENUE	RNK
Lucky	1764696	1
SR	1676247	2
CMR	1598019	3

77. Find second highest sale.

```
Select *
```

```
From (
        Select sale_id,amount,
        DENSE_RANK() OVER(order by amount desc) as rnk
        From sales
)
Where rnk=2
```

SALE_ID	AMOUNT	RNK
300	9972	2

78. Partition data by store and analyze.

```
Select st.store_name,s.sale_id,s.amount,
Sum(amount) OVER(partition by st.store_name) as store_total
From sales s
Join stores st on s.store_id=st.store_id
```

STORE_NAME	SALE_ID	AMOUNT	STORE_TOTAL
CMR	369	2515	1598019
CMR	372	5144	1598019
CMR	375	3470	1598019
CMR	378	3954	1598019
CMR	381	9424	1598019
CMR	384	5583	1598019

79. Find customer ranking per store.

```
  Select st.store_name, c.customer_name,
      Sum(s.amount) as total_spend,
      RANK() OVER (partition by st.store_name order by sum(s.amount) desc) as
rnk
  From sales s
  Join customers c on s.customer_id = c.customer_id
  Join stores st on s.store_id = st.store_id
  Group by st.store_name, c.customer_name
```

STORE_NAME	CUSTOMER_NAME	TOTAL_SPEND	RNK
CMR	Cust_2	54054	1
CMR	Cust_10	48184	2
CMR	Cust_19	45694	3
CMR	Cust_3	42384	4
CMR	Cust_33	41959	5
CMR	Cust_34	41908	6

80. Calculate frequency using window.

```
Select c.customer_name,s.sale_id,
Count(*) OVER(partition by s.customer_id) as freq from sales s
Join customers c on s.customer_id=c.customer_id
```

CUSTOMER_NAME	SALE_ID	FREQ
Cust_1	450	20
Cust_1	500	20
Cust_1	550	20
Cust_1	600	20
Cust_1	650	20
Cust_1	700	20

9. NTILE / Segmentation

Manager Requirement:
Segment customers for marketing.

Questions:

81. Divide customers into 4 segments.

```
Select c.customer_name,sum(s.amount) as tot_spend,
NTILE(4) OVER(order by sum(s.amount) desc ) as segment
From sales s
Join customers c on s.customer_id=c.customer_id
```

```
Group by c.customer_name
```

CUSTOMER_NAME	TOT_SPEND	SEGMENT
Cust_19	135669	1
Cust_36	126737	1
Cust_18	119929	1
Cust_12	116311	1
Cust_23	115118	1
Cust_29	113904	1

82. Identify top 25% customers.

```
Select *
From (
        Select c.customer_name,sum(s.amount) as tot_spend,
        NTILE(4) OVER(order by sum(s.amount) desc ) as segment
        From sales s
        Join customers c on s.customer_id=c.customer_id
        Group by c.customer_name
)
Where segment=1
```

CUSTOMER_NAME	TOT_SPEND	SEGMENT
Cust_19	135669	1
Cust_36	126737	1
Cust_18	119929	1
Cust_12	116311	1
Cust_23	115118	1
Cust_29	113904	1

83. Identify bottom 25% customers.

```
Select *
From (
        Select c.customer_name,sum(s.amount) as tot_spend,
        NTILE(4) OVER(order by sum(s.amount) desc ) as segment
        From sales s
        Join customers c on s.customer_id=c.customer_id
        Group by c.customer_name
)
```

Where segment=4

CUSTOMER_NAME	TOT_SPEND	SEGMENT
Cust_6	90030	4
Cust_49	86690	4
Cust_34	86542	4
Cust_37	85991	4
Cust_4	85346	4
Cust_48	84683	4

84. Segment stores into 3 groups.

```
Select st.store_name,sum(s.amount) as tot_spend,
NTILE(3) OVER(order by sum(s.amount) desc ) as segment
From sales s
Join stores st on s.store_id=st.store_id
Group by st.store_name
```

STORE_NAME	TOT_SPEND	SEGMENT
Lucky	1764696	1
SR	1676247	2
CMR	1598019	3

85. Create quartiles based on revenue.

```
Select c.customer_name,sum(s.amount) as tot_spend,
NTILE(4) OVER(order by sum(s.amount) desc ) as segment
From sales s
Join customers c on s.customer_id=c.customer_id
Group by c.customer_name
```

CUSTOMER_NAME	TOT_SPEND	SEGMENT
Cust_19	135669	1
Cust_36	126737	1
Cust_18	119929	1
Cust_12	116311	1
Cust_23	115118	1
Cust_29	113904	1

86. Segment customers per store.

```
Select st.store_name,c.customer_name,sum(s.amount) as tot_spend,
NTILE(3) OVER(partition by st.store_name order by sum(s.amount) desc ) as
segment
From sales s
Join customers c on s.customer_id=c.customer_id
Join stores st on s.store_id=st.store_id
Group by st.store_name,c.customer_name
```

STORE_NAME	CUSTOMER_NAME	TOT_SPEND	SEGMENT
CMR	Cust_2	54054	1
CMR	Cust_10	48184	1
CMR	Cust_19	45694	1
CMR	Cust_3	42384	1
CMR	Cust_33	41959	1
CMR	Cust_34	41908	1

87. Identify premium segment.

```
Select *
From (
        Select c.customer_name,sum(s.amount) as tot_spend,
        NTILE(4) OVER(order by sum(s.amount) desc ) as segment
        From sales s
        Join customers c on s.customer_id=c.customer_id
        Group by c.customer_name
)
Where segment=1
```

CUSTOMER_NAME	TOT_SPEND	SEGMENT
Cust_19	135669	1
Cust_36	126737	1
Cust_18	119929	1
Cust_12	116311	1
Cust_23	115118	1
Cust_29	113904	1

88. Find mid-level customers.

```
Select *
From (
        Select c.customer_name,sum(s.amount) as tot_spend,
        NTILE(4) OVER(order by sum(s.amount) desc ) as segment
        From sales s
        Join customers c on s.customer_id=c.customer_id
        Group by c.customer_name
)
Where segment in (2,3)
```

CUSTOMER_NAME	TOT_SPEND	SEGMENT
Cust_45	108584	2
Cust_41	108172	2
Cust_2	108004	2
Cust_44	107825	2
Cust_15	107671	2
Cust_33	107492	2

89. Analyze bucket distribution.

```
Select segment_label, Count(*) as customer_count
From (
  Select c.customer_name,
      Case
        When NTILE(4) OVER (order by sum(s.amount) desc) = 1 then 'Premium'
        When NTILE(4) OVER (order by sum(s.amount) desc) = 2 then 'Gold'
        When NTILE(4) OVER (order by sum(s.amount) desc) = 3 then 'Silver'
        Else 'Basic'
      End as segment_label
  From sales s
  Join customers c on s.customer_id = c.customer_id
  Group by c.customer_name
)
Group by segment_label
```

```
Order by segment_label;
```

SEGMENT_LABEL	CUSTOMER_COUNT
Basic	12
Gold	13
Premium	13
Silver	12

90. Create marketing segments.

```
Select c.customer_name,
    Case
      When NTILE(4) OVER (order by sum(s.amount) desc) = 1 then 'Premium'
      When NTILE(4) OVER (order by sum(s.amount) desc) = 2 then 'Gold'
      When NTILE(4) OVER (order by sum(s.amount) desc) = 3 then 'Silver'
      Else 'Basic'
    End as segment_label
  From sales s
  Join customers c on s.customer_id = c.customer_id
  Group by c.customer_name
```

CUSTOMER_NAME	SEGMENT_LABEL
Cust_19	Premium
Cust_36	Premium
Cust_18	Premium
Cust_12	Premium
Cust_23	Premium
Cust_29	Premium

10. FIRST_VALUE / LAST_VALUE

Manager Requirement:
Compare performance with best and worst.

Questions:

91. Find highest sale.

```
Select sale_id,amount,
FIRST_VALUE(amount) OVER(order by amount desc) as highest_sale
```

From sales

SALE_ID	AMOUNT	HIGHEST_SALE
408	159	9975
409	7623	9975
410	8404	9975
411	2346	9975
412	5041	9975
413	2192	9975

92. Find lowest sale.

Select sale_id,amount,
LAST_VALUE(amount) OVER(order by amount desc ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as lowest_sale
From sales

SALE_ID	AMOUNT	LOWEST_SALE
408	159	102
409	7623	102
410	8404	102
411	2346	102
412	5041	102

93. Compare each sale with highest.

Select sale_id,amount,
FIRST_VALUE(amount) OVER(order by amount desc) as highest_sale,
Amount-FIRST_VALUE(amount) OVER(order by amount desc) as diff
From sales

SALE_ID	AMOUNT	HIGHEST_SALE	DIFF
408	159	9975	-9816
409	7623	9975	-2352
410	8404	9975	-1571
411	2346	9975	-7629
412	5041	9975	-4934

94. Compare each sale with lowest.

Select sale_id,amount,
LAST_VALUE(amount) OVER(order by amount desc ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as lowest_sale,
Amount -LAST_VALUE(amount) OVER(order by amount desc ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as diff
From sales

SALE_ID	AMOUNT	LOWEST_SALE	DIFF
408	159	102	57
409	7623	102	7521
410	8404	102	8302
411	2346	102	2244
412	5041	102	4939
413	2192	102	2090

95. Find highest sale per store.

Select st.store_name,s.sale_id,s.amount,
FIRST_VALUE(s.amount) OVER(partition by st.store_name order by s.amount desc) as highest_sale from sales s
Join stores st on s.store_id=st.store_id

STORE_NAME	SALE_ID	AMOUNT	HIGHEST_SALE
CMR	369	2515	9972
CMR	372	5144	9972
CMR	375	3470	9972
CMR	378	3954	9972
CMR	381	9424	9972
CMR	384	5583	9972

96. Find lowest sale per store.

Select st.store_name,s.sale_id,s.amount,
LAST_VALUE(amount) OVER(partition by st.store_name order by amount desc ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as lowest_sale
from sales s

Join stores st on s.store_id=st.store_id

STORE_NAME	SALE_ID	AMOUNT	LOWEST_SALE
CMR	369	2515	111
CMR	372	5144	111
CMR	375	3470	111
CMR	378	3954	111
CMR	381	9424	111
CMR	384	5583	111

97. Compare category performance.

```
Select p.category,s.amount,
FIRST_VALUE(s.amount) OVER(partition by p.category order by s.amount desc) as
best_sale,
LAST_VALUE(s.amount) OVER(partition by p.category order by s.amount desc
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as
worst_sale
From sales s
Join products p on s.product_id=p.product_id
```

CATEGORY	AMOUNT	BEST_SALE	WORST_SALE
Electronics	8888	9972	102
Electronics	3099	9972	102
Electronics	9458	9972	102
Electronics	1177	9972	102
Electronics	1541	9972	102

98. Find best customer per store.

```
Select *
From (
  Select st.store_name, c.customer_name,
      Sum(s.amount) as total_spend,
      RANK() OVER (partition by st.store_name order by sum(s.amount) desc) as
rnk
  From sales s
  Join customers c on s.customer_id = c.customer_id
  Join stores st on s.store_id = st.store_id
  Group by st.store_name, c.customer_name
)
Where rnk = 1;
```

STORE_NAME	CUSTOMER_NAME	TOTAL_SPEND	RNK
CMR	Cust_2	54054	1
Lucky	Cust_12	59877	1
SR	Cust_45	49139	1

99. Find worst performing category.

```
Select*
From (
  Select p.category,
      Sum(s.amount) as total_revenue,
      RANK() OVER (order by sum(s.amount)) as rnk
  From sales s
  Join products p on s.product_id = p.product_id
  Group by p.category
)
Where rnk = 1;
```

CATEGORY	TOTAL_REVENUE	RNK
Fashion	1174591	1

100. Analyze gap between best and worst.

```
Select sale_id,amount,
FIRST_VALUE(amount) OVER(order by amount desc) as max_sale,
LAST_VALUE(amount) OVER(order by amount desc ROWS BETWEEN
UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as min_sale,
FIRST_VALUE(amount) OVER(order by amount desc)-
LAST_VALUE(amount) OVER(order by amount desc ROWS BETWEEN
UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) as gap
From sales
```

SALE_ID	AMOUNT	MAX_SALE	MIN_SALE	GAP
408	159	9975	102	9873
409	7623	9975	102	9873
410	8404	9975	102	9873
411	2346	9975	102	9873
412	5041	9975	102	9873
413	2192	9975	102	9873

11. Date Analysis

Manager Requirement:
Analyze time-based trends.

Questions:

101. Find daily sales.

```
Select sale_date,sum(amount) from sales
Group by sale_date
```

SALE_DATE	SUM(AMOUNT)
2/18/2025, 12:00:0(	50129
2/19/2025, 12:00:0(	78520
2/20/2025, 12:00:0(	64755
2/21/2025, 12:00:0(	64797
2/22/2025, 12:00:0(	63524
2/23/2025, 12:00:0(	49387

102. Find monthly sales.

```
Select to_char(sale_date,'YYYY-MM') as month,
Sum(amount) from sales
Group by to_char(sale_date,'YYYY-MM')
```

MONTH	SUM(AMOUNT)
2025-02	1590879
2025-03	1689433
2025-01	1758650

103. Find weekend sales.

```
Select sale_date,sum(amount) as weekend_sales from sales
Where to_char(sale_date,'dy') in ('sat','sun')
Group by sale_date
```

Order by sale_date

SALE_DATE	WEEKEND_SALES
1/4/2025, 12:00:00	52615
1/5/2025, 12:00:00	60661
1/11/2025, 12:00:0(	48701
1/12/2025, 12:00:0(	74816
1/18/2025, 12:00:0(	55076
1/19/2025, 12:00:0(	36392

104. Find first sale date.

Select min(sale_date) as first_sale from sales

FIRST_SALE
1/1/2025, 12:00:00

105. Find last sale date.

Select max(sale_date) as last_sale from sales

LAST_SALE
3/31/2025, 12:00:0(

106. Find sales in specific month.

Select * from sales
Where to_char(sale_date,'YYYY-MM')='2025-02'

SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
408	9	1	1	2/18/2025, 12:00:0(	159	4	(null)
409	10	2	2	2/19/2025, 12:00:0(	7623	5	27
410	11	3	3	2/20/2025, 12:00:0(	8404	1	351
411	12	1	4	2/21/2025, 12:00:0(	2346	2	53
412	13	2	1	2/22/2025, 12:00:0(	5041	3	(null)
413	14	3	2	2/23/2025, 12:00:0(	2192	4	240

107. Find sales growth month-wise.

```
Select month,total_sales,
Total_sales-LAG(total_sales) OVER(order by month) as growth
From (
        Select to_char(s.sale_date,'YYYY-MM') as month,
        Sum(s.amount) as total_sales
        From sales s
```

```
                    Group by to_char(s.sale_date,'YYYY-MM')
)
```

MONTH	TOTAL_SALES	GROWTH
2025-01	1758650	*(null)*
2025-02	1590879	-167771
2025-03	1689433	98554

108. Find inactive customers (30 days).

```
select c.customer_name from customers c
left join sales s
on c.customer_id=s.customer_id
group by c.customer_name
having max(s.sale_date)<SYSDATE-30 or max(s.sale_date) is null
```

CUSTOMER_NAME
Cust_9
Cust_10
Cust_11
Cust_12
Cust_13
Cust_14

109. Find repeat customers.

```
select c.customer_name,count(*) from sales s
join customers c on s.customer_id=c.customer_id
group by c.customer_name
having count(*)>1
```

CUSTOMER_NAME	COUNT(*)
Cust_9	20
Cust_10	20
Cust_11	20
Cust_12	20
Cust_13	20
Cust_14	20

110. Find sales trend.

Select sale_date,sum(amount) OVER(order by sale_date) as trend
From sales

SALE_DATE	TREND
1/1/2025, 12:00:00	62574
1/1/2025, 12:00:00	62574
1/1/2025, 12:00:00	62574
1/1/2025, 12:00:00	62574
1/1/2025, 12:00:00	62574
1/1/2025, 12:00:00	62574